

Advanced Algorithms for Discrete Optimization

Models, Paradigms, and Algorithms

Pierre Schaus

ICTEAM Institute
UCLouvain, Belgium



September 17, 2026

© 2026 Pierre Schaus

Advanced Algorithms for Discrete Optimization: Models, Paradigms, and Algorithms

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without prior written permission of the author.

Published with L^AT_EX and biber.

First edition, September 17, 2026

Preface

Discrete optimization lies at the core of computer science, operations research, and applied mathematics. It tackles problems where decision variables take values from discrete domains—such as selecting schedules, routing vehicles, allocating scarce resources, or configuring communication networks.

While many discrete optimization problems are computationally intractable in the worst case (**NP-hard**), real-world applications demand fast, robust, and often provably optimal solutions. To meet this challenge, several complementary algorithmic paradigms have been developed over the last several decades:

- **Dynamic Programming:** Decomposing problems into overlapping subproblems governed by the principle of optimality.
- **Branch and Bound:** Systematically enumerating the solution space while aggressively pruning non-promising regions through relaxation bounds.
- **Linear Programming & Duality:** Exploiting geometric and algebraic structures of polyhedra to find exact solutions or compute strong bounds.
- **Lagrangian Relaxation:** Dualizing complicating constraints to decompose hard problems into tractable subproblems.
- **Local Search & Metaheuristics:** Exploring neighborhood structures to quickly uncover high-quality heuristic solutions in massive search spaces.
- **Constraint Programming:** Leveraging declarative modeling, domain filtering, and constraint propagation to prune infeasible regions efficiently.
- **Benchmarking Optimization Algorithms:** Rigorous empirical evaluation methodologies, shifted geometric means, and Dolan-Moré performance profiles.

Audience and Prerequisites

This book is designed for advanced graduate students, doctoral researchers, and practitioners who want to master both theoretical foundations and practical implementation of modern discrete optimization techniques. Readers are expected to have a solid background in data structures, analysis of algorithms, and basic linear algebra.

Structure of the Book

Each chapter is dedicated to one foundational paradigm. Chapters feature formal definitions, theoretical properties, pseudocode for key algorithms, concrete examples, and exercises.

Pierre Schaus

Louvain-la-Neuve, Belgium
September 17, 2026

Contents

Preface	iii
1 Introduction to Discrete Optimization	1
1.1 From Classical Polynomial Algorithms to the Reality of NP-Hardness	1
1.2 Industrial Applications: Where Discrete Optimization Drives Value	2
1.2.1 Vehicle Routing and Fleet Logistics (VRP)	2
1.2.2 Complex Workforce and Industrial Scheduling	2
1.2.3 Cloud Computing and Resource Allocation	3
1.2.4 Telecommunications and Computer Network Traffic Engineering	3
1.3 The Practitioner’s Attitude: Tackling NP-Hard Problems	3
1.4 The Diagnostic Compass: Two Fundamental Questions	4
1.5 The Art of Modeling: Decisions, Trade-Offs, and Imperfect Data	5
1.5.1 Identifying High-Leverage Decision Variables	5
1.5.2 The Myth of Exact Optimality with Imperfect Data	5
1.6 Formal Problem Formulation	6
1.7 A Taxonomy of Algorithmic Paradigms	7
1.8 Bridging Theory and Practice: Generic Solvers and Hands-On Projects	7
1.8.1 Architecting Generic Optimization Engines	7
1.8.2 Hands-On Learning: Six Programming Projects	8
1.8.3 Language Selection: Why Java?	9
1.9 Exercises	10
2 Dynamic Programming	13
2.1 Computational Complexity: NP-Completeness of Knapsack	13
2.1.1 The Polynomial-Time Reduction Technique	14
2.2 Bellman Recurrence Equations for Knapsack	15
2.3 Naive Recursive Implementation and Time Complexity	16
2.3.1 Complexity Analysis of the Naive Recursion	16
2.4 Overlapping Subproblems and Memoization	16
2.4.1 Memoization with a Hash Map	17
2.5 Tabulation: Implementation with a 2D Array	18
2.5.1 Iterative Algorithm (Bottom-Up)	18
2.5.2 Subproblem Dependencies: The Two Incoming Edges	19
2.5.3 The Tabulation Matrix Illustrated	20
2.5.4 Reconstructing the Optimal Solution via Backtracking	21
2.5.5 The Tabulation Grid as a DAG: The Longest Path Perspective	22
2.6 Time Complexity and Pseudo-Polynomial Time	23
2.6.1 Is this Algorithm Polynomial?	23

2.6.2	Weak vs. Strong NP-Hardness	23
2.7	Case Study: The Range Partitioning Problem	24
2.7.1	Dynamic Programming Formulation	24
2.7.2	Prefix Sum Precomputation	25
2.7.3	Java Implementation	25
2.8	Dynamic Programming as Paths in a Directed Acyclic Graph	26
2.9	A Generic Dynamic Programming Framework in Java	27
2.9.1	The State Abstraction	27
2.9.2	The Transition Class	28
2.9.3	The Model Abstraction	28
2.9.4	The Generic Solver Engine	28
2.10	Instantiating Knapsack in the Generic Framework	30
2.10.1	The Knapsack State	30
2.10.2	The Knapsack Model	30
2.11	Exercises	32
3	Branch and Bound	33
4	Linear Programming and Polyhedral Theory	35
5	Lagrangian Relaxation	37
6	Local Search and Metaheuristics	39
7	Constraint Programming	41
8	Benchmarking Optimization Algorithms	43
	References	45

Introduction to Discrete Optimization

Discrete optimization is the science and engineering of making optimal decisions from a finite or countably infinite set of possibilities. It stands at the intersection of computer science, operations research, and applied mathematics, providing the algorithmic backbone for managing complex modern infrastructure, logistics networks, manufacturing supply chains, and computing systems.

Unlike continuous optimization, where calculus, gradients, and smooth geometric structures guide search toward local or global extrema, discrete optimization operates on combinatorial landscapes. In these landscapes, moving a single discrete decision variable from 0 to 1 can drastically alter the feasibility and quality of an entire system.

1.1 From Classical Polynomial Algorithms to the Reality of NP-Hardness

In introductory courses on data structures and algorithms, students are introduced to a collection of celebrated polynomial-time algorithms:

- **Dijkstra’s and Bellman-Ford’s algorithms** for single-source shortest paths on graphs in $\mathcal{O}(|E| + |V| \log |V|)$ time.
- **Kruskal’s and Prim’s algorithms** for minimum spanning trees in $\mathcal{O}(|E| \log |V|)$ time.
- **Edmonds-Karp and push-relabel algorithms** for maximum network flow and bipartite matching in polynomial time.
- **Divide-and-conquer sorting** operating in $\mathcal{O}(n \log n)$ time.

These classical algorithms are elegant and practical. However, they share a common trait: they solve problems characterized by very specific algebraic or matroid structures that prevent combinatorial explosion.

When graduating from textbook abstractions to real-world industrial systems, a sobering truth emerges: **almost every real-world discrete optimization problem is NP-hard**. Consider what happens when we introduce a single realistic constraint into the classical polynomial problems:

- Find a shortest path between two nodes in a network: *Solvable in polynomial time* by Dijkstra’s algorithm.
- Find a shortest path subject to a total transit budget or battery capacity limit (the Constrained Shortest Path Problem): *Immediately NP-hard* [GJ79].

- Find a minimum spanning tree connecting a set of terminals: *Solvable in polynomial time* by Prim’s algorithm.
- Find a minimum spanning tree where the degree of each node cannot exceed 3 (the Degree-Constrained Spanning Tree Problem): *Immediately NP-hard*.
- Match jobs to identical machines to maximize throughput: *Solvable in polynomial time* via maximum bipartite matching.
- Match jobs to machines where tasks share disjunctive resources and strict deadlines: *Immediately NP-hard*.

The combinatorial explosion is not a rare mathematical curiosity; it is the default condition of real-world decision-making.

1.2 Industrial Applications: Where Discrete Optimization Drives Value

To appreciate the reach of discrete optimization, consider the critical industrial systems that rely on the algorithms developed in this book:

1.2.1 Vehicle Routing and Fleet Logistics (VRP)

Global e-commerce and logistics networks (such as Amazon, DHL, FedEx, and UPS) dispatch hundreds of thousands of delivery vehicles daily. The canonical *Vehicle Routing Problem (VRP)* requires determining an optimal set of routes for a fleet of vehicles based at one or more depots to service a set of customers with known demands:

- **Time Windows (VRPTW):** Deliveries must arrive within customer-specified intervals (e.g., 10:00 AM – 11:30 AM).
- **Capacity and Compartment Constraints:** Vehicles have weight and volume thresholds, with refrigerated vs. non-refrigerated compartments.
- **Driver Rest and Regulations:** Routes must comply with legal driving hours and mandatory rest periods.

A fleet efficiency improvement of merely 1% in a major carrier translates into tens of millions of liters of fuel saved annually, hundreds of thousands of tons of reduced CO₂ emissions, and substantial cost reductions.

1.2.2 Complex Workforce and Industrial Scheduling

Scheduling problems allocate scarce physical resources and human personnel over time:

- **Airline Crew Pairing and Rostering:** Major airlines (such as Air France-KLM, Delta, and Lufthansa) must assign tens of thousands of pilots and flight attendants to flights over monthly horizons. Feasible schedules must satisfy hundreds of regulatory requirements from civil aviation authorities (FAA, EASA), pilot certifications, rest periods, seniority privileges, and collective bargaining agreements.
- **Semiconductor Manufacturing:** In wafer fabrication plants (fabs), expensive cluster tools and photolithography machines process microchips through hundreds of sequential stages. Optimal job sequencing must account for sequence-dependent setup times, chemical re-entry constraints, and clean-room transport delays.

- **Operating Theater Scheduling:** In healthcare, hospitals must schedule elective surgeries across limited operating rooms, coordinating surgeon availability, post-operative intensive care unit (ICU) beds, and specialized medical instrumentation under emergency disruption risks.

1.2.3 Cloud Computing and Resource Allocation

Hyper-scale cloud providers (such as Google Cloud, Microsoft Azure, and Amazon Web Services) host millions of virtual machines and containers on physical server racks across globally distributed data centers:

- **VM Placement and Consolidation:** Mapping customer workloads to physical servers to minimize total electricity and cooling power while avoiding CPU, RAM, and disk I/O bottlenecks.
- **High-Availability Constraints:** Preventing correlated failures by ensuring that primary and backup replicas of critical services are never hosted on the same physical server rack, power strip, or availability zone.

1.2.4 Telecommunications and Computer Network Traffic Engineering

Modern Internet backbones and Software-Defined Networks (SDN) deploy *Traffic Engineering (TE)* to route petabytes of data traffic across mesh topologies:

- Given a network topology graph $G = (V, E)$ with link capacities c_e and an origin-destination traffic matrix demand, compute packet paths (using Segment Routing or MPLS tunnels) that avoid link congestion, minimize end-to-end latency, and reserve failover capacity for link outages.

1.3 The Practitioner's Attitude: Tackling NP-Hard Problems

Recognizing that a problem is **NP-hard** is the beginning of the algorithm designer's task, not the end of it. **NP-hardness does not mean that a problem cannot be solved.**

Theoretical complexity analysis is inherently pessimistic: it characterizes the asymptotic computational effort required to solve the *pathological worst-case instance* among all conceivable inputs. However:

1. Real-world instances rarely resemble worst-case adversarial constructions. They exhibit geographic clustering, sparse interactions, metric triangle inequalities, and structured demand distributions.
2. Small- and medium-sized instances of **NP-hard** problems can routinely be solved to **proven mathematical optimality** using exact methods such as Branch-and-Bound, Branch-and-Cut, Dynamic Programming, and Constraint Programming.
3. Massive industrial instances with millions of variables can be solved to within fractions of a percent of optimality within minutes using decompositions, relaxations, and Large Neighborhood Search heuristics.

Example 1.1 Exact Solvability at Scale: The Concorde TSP Solver

A striking demonstration of exact optimization in action is the celebrated *Concorde TSP Solver*, developed by David Applegate, Robert Bixby, Vašek Chvátal, and William Cook [App+06].

Although the Traveling Salesperson Problem on n vertices has $(n - 1)!/2$ possible tours—an astronomically vast search space even for modest n —Concorde routinely solves instances of several hundred to a few thousand cities (such as the standard TSPLIB benchmarks with 1,000 to 2,000 cities) to **proven mathematical optimality in seconds or minutes** on an ordinary workstation.

Even more remarkably, Concorde holds the world record for the largest TSP instance solved to proven mathematical optimality: the benchmark instance **pla85900**, comprising **85,900 cities** derived from a programmable logic array circuit layout at Bell Laboratories.

Concorde achieves this through a state-of-the-art *Branch-and-Cut* framework: tightly coupling linear programming relaxations, iterative separation of facet-defining cutting planes (subtour elimination constraints, comb inequalities, and clique trees), primal metaheuristics (the Chained Lin-Kernighan algorithm) to uncover near-optimal upper bounds early, and branch-and-bound search. This achievement demonstrates that **NP**-hardness is not an insurmountable barrier: with deep polyhedral insight and algorithmic engineering, exact optimization routinely conquers instances of immense practical scale.

1.4 The Diagnostic Compass: Two Fundamental Questions

When confronted with a new discrete optimization problem in academic research or industrial practice, the seasoned practitioner does not immediately open a solver or write code. Instead, they orient their algorithmic strategy by answering two fundamental questions:

The Two Diagnostic Questions of Discrete Optimization

1. What size is your problem?

- How many discrete decision variables are involved? (Tens? Thousands? Millions?)
- What is the planning horizon and constraint density? (A single warehouse with 10 vehicles, or a continental supply chain with 50,000 stops?)

2. How much time do you give me to solve it?

- *Real-Time Online Operation*: Decisions must be returned in milliseconds or a few seconds (e.g., dynamic dispatch in ride-hailing, real-time packet rerouting).
- *Operational Short-Turnaround*: Solutions needed within 1 to 15 minutes (e.g., flight gate re-assignment during an airport storm, morning truck dispatch).
- *Tactical and Strategic Offline Planning*: Overnight or multi-hour computational budget (e.g., monthly pilot rostering, annual warehouse location).

The answers to these two questions determine which algorithmic paradigm—or combination of paradigms—should be deployed:

Table 1.1: Algorithmic Paradigm Selection Matrix based on Problem Scale and Time Budget.

Problem Scale	Generous Time Budget (Hours)	Tight Time Budget (Seconds to Minutes)
Small	Exact Solvers: Mixed-Integer Programming (MIP), Constraint Programming (CP), Branch-and-Bound. <i>Result: Provably optimal solution.</i>	Dynamic Programming / Fast MIP: Table lookup, prefix evaluation, or fast tree search with early cutoff. <i>Result: Optimal or near-optimal.</i>
Medium	Decompositions: Lagrangian Relaxation, Column Generation, Branch-and-Cut. <i>Result: Proven optimality or $< 0.1\%$ optimality gap.</i>	Constraint Propagation + Heuristics: CP with custom variable selection, restricted tree search. <i>Result: High-quality feasible solutions.</i>
Massive	Hybrid Matheuristics: Large Neighborhood Search (LNS) combined with MIP/CP subproblem repair. <i>Result: Deeply optimized solution with strong dual bounds.</i>	Metaheuristics: Tabu Search, Simulated Annealing, greedy randomized adaptive search (GRASP). <i>Result: Fast, robust heuristic solution.</i>

1.5 The Art of Modeling: Decisions, Trade-Offs, and Imperfect Data

Solving a discrete optimization problem is not merely a mechanical application of algorithms; it is an **engineering art**. A sophisticated algorithm running on a poorly conceived model will fail, whereas a clean, insightful formulation can solve in seconds on standard solvers.

1.5.1 Identifying High-Leverage Decision Variables

The foremost task of the modeler is to discern which decisions provide high leverage and business value:

- Should we model the exact microsecond when a truck engine ignites, or simply the sequence of delivery stops and time windows?
- Should a railway dispatching model schedule every individual switch point and signal light, or model blocks of track between mainline stations?

There is an inevitable tension between **model fidelity** (capturing every operational nuance) and **computational tractability** (ensuring the model can be solved within the allotted time). The art of modeling lies in abstracting secondary details while rigorously preserving the constraints that dictate system feasibility.

1.5.2 The Myth of Exact Optimality with Imperfect Data

In academic theory, an algorithm designer strives to prove mathematical optimality down to an optimality gap of 0.00%. In real-world industrial practice, however, **obsessing over exact optimality on imperfect data is an illusion of precision**.

Consider two illustrative industrial realities:

1. **Industrial Manufacturing and Project Scheduling:** A mathematical model treats task durations as fixed integers (e.g., task *A* takes exactly 45 minutes). In a physical factory, however, task durations vary due to worker fatigue, micro-stoppages, machine calibration drifts, and raw material variations. A mathematically “optimal” schedule

that packs jobs with zero buffer time will collapse the moment a task runs 3 minutes late. A slightly sub-optimal schedule with robust time buffers is practically superior to an optimal brittle schedule.

2. **Traffic Engineering in Computer Networks:** Network routing optimization computes paths assuming that the traffic demand between each origin and destination is known and summarized in a traffic matrix. In practice, a traffic matrix is only an *estimation* extrapolated from historical telemetry counters (such as NetFlow or IPFIX samples). Actual network traffic fluctuates stochastically with video streaming bursts, software downloads, and unexpected link outages. Spending eight hours to find a routing scheme that is 0.05% better on yesterday's estimated traffic matrix is useless if today's traffic shifts by 10%.

When input data contains 5% uncertainty, spending hours of computational effort to close the final 0.1% optimality gap is an ineffective use of resources. What matters in practice is:

- **Robustness:** Solutions remain feasible and high-performing under modest real-world perturbations.
- **Speed of Response:** Delivering actionable solutions within operational decision cycles.
- **Transparent Constraints:** Providing operators with clear explanations of bottlenecks and trade-offs.

1.6 Formal Problem Formulation

Formally, a discrete optimization problem is defined as follows:

Definition 1.1 *Combinatorial Optimization Problem*

A *combinatorial optimization problem* is a tuple $(\mathcal{S}, \mathcal{X}, f)$, where:

- $\mathcal{S}$ is the configuration space, typically $\mathcal{S} \subseteq \mathbb{Z}^n$ or $\mathcal{S} \subseteq \{0, 1\}^n$,
- $\mathcal{X} \subseteq \mathcal{S}$ is the set of *feasible solutions* satisfying a finite collection of constraints $g_j(\mathbf{x}) \leq 0$ for $j \in \{1, \dots, m\}$,
- $f : \mathcal{S} \rightarrow \mathbb{R}$ is the *objective function*.

The goal is to find an optimal solution $\mathbf{x}^* \in \mathcal{X}$ such that:

$$f(\mathbf{x}^*) = \min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad (\text{for minimization}), \quad \text{or} \quad f(\mathbf{x}^*) = \max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) \quad (\text{for maximization}). \quad (1.1)$$

- If $\mathcal{X} = \emptyset$, the problem is **infeasible** (no configuration satisfies all constraints simultaneously).
- If there exist feasible solutions $\mathbf{x} \in \mathcal{X}$ with $f(\mathbf{x}) \rightarrow -\infty$ (for minimization), the problem is **unbounded**.
- Any configuration $\mathbf{x} \in \mathcal{X}$ is a **feasible solution**; its objective value $f(\mathbf{x})$ provides a valid *primal bound* on the global optimum $f(\mathbf{x}^*)$.

Theorem 1.1 *Combinatorial Explosion of Binary Spaces*

Let $\mathcal{S} = \{0,1\}^n$ be the space of binary decisions on n independent choices. The cardinality of the search space is:

$$|\mathcal{S}| = 2^n. \quad (1.2)$$

Adding a single variable doubles the size of the search space. For $n = 100$, $|\mathcal{S}| = 2^{100} \approx 1.27 \times 10^{30}$, a number exceeding the estimated number of grains of sand on Earth.

1.7 A Taxonomy of Algorithmic Paradigms

To navigate this combinatorial explosion, this book covers the seven core pillars of modern discrete optimization:

1. **Dynamic Programming (Chapter 2)**: Recursive decomposition into overlapping subproblems governed by Bellman’s principle of optimality, DAG representations, and generic solver architectures.
2. **Branch and Bound (Chapter 3)**: Systematic state-space search over decision trees, employing mathematical relaxations and dominance bounds to prune non-promising subtrees.
3. **Linear and Mixed-Integer Programming (Chapter 4)**: Continuous relaxations, polyhedral combinatorics, extreme point geometry, duality theory, and cutting plane generation.
4. **Lagrangian Relaxation (Chapter 5)**: Decomposition of hard problems into tractable components by pricing out complicating constraints via subgradient optimization.
5. **Local Search & Metaheuristics (Chapter 6)**: Neighborhood structure exploration, trajectory methods (Simulated Annealing, Tabu Search), and Large Neighborhood Search (LNS) for rapid discovery of near-optimal solutions in massive spaces.
6. **Constraint Programming (Chapter 7)**: Declarative reasoning over finite variable domains, consistency notions, filtering algorithms for global constraints, and propagation engines.
7. **Benchmarking Optimization Algorithms (Chapter 8)**: Scientific methodology for empirical performance evaluation, shifted geometric means, Dolan-Moré performance profiles, and standard problem repositories.

1.8 Bridging Theory and Practice: Generic Solvers and Hands-On Projects

A defining philosophy of this course and textbook is that discrete optimization cannot be truly mastered solely through mathematical equations on paper or through one-off, hardcoded scripts. To develop the algorithmic intuition and problem-solving skills of an expert, one must bridge rigorous theoretical concepts with clean, robust, and extensible software architectures.

1.8.1 Architecting Generic Optimization Engines

In introductory computer science, dynamic programming or tree search algorithms are frequently presented as rigid, problem-specific routines—such as a hardcoded 2D array and nested loops for the 0/1 Knapsack problem. While such implementations demonstrate the

recurrence relation, they fail to provide a reusable foundation for tackling new, unfamiliar combinatorial challenges.

In industrial operations research and advanced software engineering, practitioners do not reinvent search algorithms from scratch for each new application. Instead, they design and utilize **generic solver frameworks** governed by clean declarative abstractions:

- **Generic Dynamic Programming:** Decoupling the problem topology (states, transitions, rewards) from the search engine (memoization, topological traversal, and solution reconstruction). The same generic engine that solves Knapsack can solve the Traveling Salesperson Problem or shortest path problems without modifying a single line of search logic.
- **Generic Branch-and-Bound Engines:** Providing reusable search tree explorations, customizable node selection strategies, priority queues, and dual bounding interfaces independent of the concrete problem formulation.
- **Constraint Propagation Engines:** Implementing variable domain representations, dependency graphs, and event-driven filtering loops that execute independently of specific global constraints.
- **Metaheuristic and Local Search Architectures:** Designing modular neighborhood operators, accept/reject evaluators, and trajectory managers (e.g., Tabu Search and Large Neighborhood Search).

Throughout this textbook, every major algorithmic paradigm is presented alongside its generic software architecture, demonstrating how mathematical principles translate into elegant, production-grade tools.

1.8.2 Hands-On Learning: Six Programming Projects

To ground these theoretical abstractions in practice, the reader is invited to complete and extend some of these generic optimization tools across **six structured programming projects**:

1. **Project 1: Dynamic Programming:** Completing the core search and solution reconstruction engine (`getValueForState`, `getSolution`, `rebuildSolution`) in the generic parameterized DP framework (`Model<S>`, `State`, `Transition<S>`) in package `dynamicprogramming`. Validating the engine on 0/1 Knapsack, and then modeling and solving the Traveling Salesperson Problem (Held-Karp algorithm) using bitset state encodings (`TSPState`).
2. **Project 2: Branch-and-Bound and Lagrangian Relaxation:** Tackling the TSP with tree search and mathematical relaxations in package `branchandbound`. Implementing a simple 1-tree lower bound (`SimpleOneTree`), constructing a Branch-and-Bound solver (`BranchAndBoundTSP`), and deriving an enhanced dual bound reinforced by Lagrangian relaxation (`HeldKarpOneTree`) using subgradient optimization to penalize vertex degree violations.
3. **Project 3: Linear Programming, Simplex, and Maximum Flows:** Formulating combinatorial problems into standard linear programs ($\max cx$ s.t. $Ax \leq b, x \geq 0$) in package `linearprogramming`. Generating constraint matrices for Maximum Flow (`FlowMatrices`) and Maximum Bipartite Matching (`MatchingMatrices`), and implementing the Big-M Simplex method (`BigMSimplex`) to handle initial infeasibilities via lexicographic single-phase optimization, evaluated on Stigler's Diet problem.
4. **Project 4: Local Search and Invariants:** Developing a local search metaheuristic in

`localsearch` for the Pigment Sequencing Problem (PSP), a discrete lot-sizing problem with sequence-dependent changeovers and deadlines. Implementing $\mathcal{O}(1)$ incremental cost invariants (`StockingCostInvariant`, `ChangeoverCostInvariant`), feasibility checks, restarts, and swap neighborhood exploration.

5. **Project 5: Constraint Programming:** Building core engine and filtering components for the TinyCSP solver in package `constraintprogramming`. Implementing domain reduction operations (`removeAbove`, `removeBelow`) and bound-consistent filtering algorithms for global constraints (`Sum` and `LessOrEqual`), and modeling complex combinatorial puzzles such as the Magic Square and Killer Sudoku.
6. **Project 6: Dynamic Programming with A^* Search:** Unifying dynamic programming state spaces with heuristic search algorithms. Formulating a DP model for the Traveling Salesperson Problem and implementing admissible heuristic evaluation functions $h(s)$ inspired by Branch-and-Bound bounds. Implementing and benchmarking A^* search variants in package `astar`: standard A^* , Weighted A^* ($w_h \in \{1.25, 1.5, 2.0\}$) to trade bounded sub-optimality for search speed, and Anytime Weighted A^* to iteratively deliver and refine solutions under tight computational time limits.

All project starter code, automated unit test suites, benchmark problem instances, and validation harnesses are openly available in the accompanying GitHub repository:

<https://github.com/pschaus/linfo2266-students>

The internal mechanisms of these codebases are explained progressively throughout the chapters, directly accompanying the theoretical development of each topic.

1.8.3 Language Selection: Why Java?

A natural question arises: why is **Java** chosen as the primary programming language for this book and its projects, rather than C++ or Python?

In commercial operations research and high-performance solver development, low-level languages such as **C** and **C++** have historically been dominant. Commercial solvers (such as Gurobi, CPLEX, or Concorde) are predominantly implemented in C or C++ to exploit direct pointer manipulation, manual memory management, and bespoke cache layouts.

However, for a Master’s level curriculum and for building deep algorithmic intuition, C++ presents significant pedagogical drawbacks:

- **Excessive Cognitive Overhead:** Complex template metaprogramming, manual memory management, dangling pointers, segmentation faults, and platform-dependent build configurations (Make, CMake, linker errors) divert the student’s valuable cognitive energy away from discrete optimization principles toward systems-level debugging. The bar is unnecessarily high, making it harder to compile, debug, and rapidly experiment with algorithms.
- **Fragile Compilation and Toolchains:** Ensuring consistent compilation across macOS, Linux, and Windows environments when linking external C++ libraries can quickly become an unproductive hurdle.

At the opposite extreme, dynamically typed scripting languages such as **Python** offer rapid prototyping, but suffer from critical shortcomings for solver engineering:

- **Execution Speed:** The interpreted nature of Python incurs a performance penalty of $20\times$ to $100\times$ compared to compiled languages, making it impractical to execute deep tree searches with millions of nodes or high-frequency constraint propagation within acceptable time budgets.

- **Lack of Strict Static Typing:** Discrete optimization engines manipulate intricate, interconnected state graphs. Static typing is indispensable for defining clear contracts (such as `Model<S extends State>`) and catching conceptual and lifecycle bugs at compile time.

Java strikes the ideal balance between performance, safety, and readability:

1. **Strong Static Typing with Generics:** Java’s robust type system and generic polymorphism enable clean, expressive abstractions for states, decisions, and models, allowing students to design generic solvers with ironclad compile-time safety.
2. **Sweet Spot Between Execution Speed and Readability:** Modern Java Virtual Machines (JVM) feature sophisticated HotSpot Just-In-Time (JIT) compilers, delivering execution throughput that is frequently within a factor of $1.5\times$ to $2\times$ of optimized C++ for memory-bound combinatorial algorithms, while maintaining clean, legible code.
3. **Frictionless Developer Experience:** Automatic garbage collection eliminates the danger of dangling pointers and memory leaks during complex tree traversals. Furthermore, Java guarantees seamless cross-platform portability, first-class IDE tooling, straightforward compilation, informative stack traces, and standardized unit testing with JUnit.

This pedagogical and engineering rationale mirrors the design philosophy of the seminal textbook *Algorithms* by Robert Sedgewick and Kevin Wayne [SW11], which famously adopted Java to teach foundational data structures and graph algorithms. By using Java, we ensure that students and practitioners spend their time thinking about state spaces, bounding functions, pruning theorems, and filtering logic—while still constructing production-grade tools that run with genuine industrial speed.

1.9 Exercises

Exercise 1.1 *Knapsack Formulation*

Formulate the 0/1 Knapsack problem with items $I = \{0, \dots, n - 1\}$, values $v_i > 0$, weights $w_i > 0$, and capacity $C > 0$ as a discrete optimization problem according to Definition 1.1. Explicitly specify the configuration space $\mathcal{S}$, the constraint defining $\mathcal{X}$, and the objective function f .

Exercise 1.2 *Diagnostic Compass for Industrial Problems*

For each of the following industrial scenarios, answer the two diagnostic questions (problem size and available time budget) and recommend an appropriate algorithmic paradigm from Table 1.1:

1. A ride-hailing platform matching 50 passengers to 50 nearby drivers every 3 seconds.
2. An airline creating a monthly crew schedule for 3,000 pilots across 15,000 flight legs over a weekend run.
3. An e-commerce warehouse routing 12 automated guided vehicles (AGVs) to pick 200 customer orders before a 2:00 PM shipping cutoff.

Exercise 1.3 *Modeling Trade-offs in Network Routing*

In traffic engineering, explain why modeling every individual TCP packet sequence number makes the routing optimization computationally impossible. What aggregated flow variables are used instead, and what operational risk does this abstraction introduce?

Dynamic Programming

Dynamic Programming (DP) is one of the foundational algorithmic paradigms in discrete optimization, pioneered by Richard Bellman in the 1950s [Bel57]. It provides a systematic method for solving complex optimization problems by decomposing them into a collection of overlapping subproblems, solving each subproblem once, and storing their solutions in a table or cache to avoid redundant work.

Before diving into the mechanics of dynamic programming, it is crucial to understand the computational complexity of the problems we aim to solve. We begin with a rigorous reminder of the **NP**-completeness of the 0/1 Knapsack problem.

2.1 Computational Complexity: NP-Completeness of Knapsack

In discrete optimization, the *0/1 Knapsack Problem* is formulated as follows: given a set of n items $I = \{0, \dots, n-1\}$, where each item i has an integer profit $v_i > 0$ and an integer weight $w_i > 0$, and a knapsack with maximum weight capacity $C \in \mathbb{Z}^+$, select a subset of items to maximize the total profit without exceeding the capacity:

$$\begin{aligned} & \text{maximize} && \sum_{i \in I} v_i x_i \\ & \text{subject to} && \sum_{i \in I} w_i x_i \leq C \\ & && x_i \in \{0, 1\}, \quad \forall i \in I. \end{aligned} \tag{2.1}$$

To analyze its computational complexity from the perspective of complexity theory [GJ79], we consider its decision version:

Definition 2.1 *0/1 Knapsack Decision Problem (KNAPSACK)*

Given item values $v_0, \dots, v_{n-1} \in \mathbb{Z}^+$, weights $w_0, \dots, w_{n-1} \in \mathbb{Z}^+$, capacity $C \in \mathbb{Z}^+$, and a target profit threshold $K \in \mathbb{Z}^+$, does there exist a binary assignment $\mathbf{x} \in \{0, 1\}^n$ such that:

$$\sum_{i=0}^{n-1} w_i x_i \leq C \quad \text{and} \quad \sum_{i=0}^{n-1} v_i x_i \geq K ? \tag{2.2}$$

2.1.1 The Polynomial-Time Reduction Technique

To prove that a decision problem B is **NP**-complete, one must follow the standard reduction methodology [Kar72]:

1. **Membership in NP:** Show that $B \in \mathbf{NP}$ by providing a polynomial-time verifier that takes an instance and a candidate certificate (witness) and checks its validity in polynomial time.
2. **Hardness by Reduction:** Choose a known **NP**-complete problem A and construct a function $f : \text{Instances}(A) \rightarrow \text{Instances}(B)$ such that:
 - f is computable in polynomial time in the size of the instance of A ($f \in \mathbf{P}$).
 - f preserves the answer: an instance x of A is a YES-instance if and only if $f(x)$ is a YES-instance of B :

$$x \in A \iff f(x) \in B. \quad (2.3)$$

Definition 2.2 The Subset-Sum Problem (SUBSET-SUM)

Given a multiset of positive integers $A = \{a_0, a_1, \dots, a_{n-1}\}$ and a target integer $T \in \mathbb{Z}^+$, does there exist a subset of indices $S \subseteq \{0, \dots, n-1\}$ such that:

$$\sum_{i \in S} a_i = T ? \quad (2.4)$$

The SUBSET-SUM problem is one of Karp's 21 classic **NP**-complete problems [Kar72].

Theorem 2.1 NP-Completeness of 0/1 Knapsack

The KNAPSACK decision problem is **NP**-complete.

Proof. First, we show that KNAPSACK $\in \mathbf{NP}$. Given a candidate solution $\mathbf{x} = (x_0, \dots, x_{n-1}) \in \{0, 1\}^n$, a verifier computes $\sum_{i=0}^{n-1} w_i x_i$ and $\sum_{i=0}^{n-1} v_i x_i$, and checks whether the total weight is $\leq C$ and the total profit is $\geq K$. This requires $\mathcal{O}(n)$ additions and comparisons, which is polynomial in the input size. Hence, KNAPSACK $\in \mathbf{NP}$.

Second, we reduce SUBSET-SUM to KNAPSACK (SUBSET-SUM $\leq_P$ KNAPSACK). Let (A, T) be an arbitrary instance of SUBSET-SUM, where $A = \{a_0, \dots, a_{n-1}\}$ and $T \in \mathbb{Z}^+$. We construct an instance of KNAPSACK as follows:

- We create n items, with item i having weight $w_i = a_i$ and value $v_i = a_i$ for all $i \in \{0, \dots, n-1\}$.
- We set the knapsack capacity $C = T$.
- We set the target value threshold $K = T$.

This transformation requires copying n integers and setting $C = K = T$, which takes $\mathcal{O}(n)$ time.

We now establish correctness ($x \in \text{SUBSET-SUM} \iff f(x) \in \text{KNAPSACK}$):

- ($\implies$) Suppose (A, T) is a YES-instance. Then there exists a subset $S \subseteq \{0, \dots, n-1\}$ such that $\sum_{i \in S} a_i = T$. Setting $x_i = 1$ for $i \in S$ and $x_i = 0$ otherwise yields:

$$\sum_{i=0}^{n-1} w_i x_i = \sum_{i \in S} a_i = T \leq C \quad \text{and} \quad \sum_{i=0}^{n-1} v_i x_i = \sum_{i \in S} a_i = T \geq K. \quad (2.5)$$

Thus, the constructed knapsack instance is a YES-instance.

- ($\Leftarrow$) Conversely, suppose the knapsack instance is a YES-instance. Then there exists $\mathbf{x} \in \{0, 1\}^n$ such that $\sum_{i=0}^{n-1} w_i x_i \leq C = T$ and $\sum_{i=0}^{n-1} v_i x_i \geq K = T$. Because $w_i = v_i = a_i$ for each item:

$$T \leq \sum_{i=0}^{n-1} v_i x_i = \sum_{i=0}^{n-1} a_i x_i = \sum_{i=0}^{n-1} w_i x_i \leq T. \quad (2.6)$$

Squeezing between T and T forces $\sum_{i=0}^{n-1} a_i x_i = T$. The set $S = \{i \mid x_i = 1\}$ is therefore a valid subset summing to T , proving (A, T) is a YES-instance.

Since the reduction is polynomial and sound, KNAPSACK is **NP**-complete. $\square$

2.2 Bellman Recurrence Equations for Knapsack

Since KNAPSACK is **NP**-hard, exhaustive search must evaluate all 2^n possible subsets of items. For $n = 50$, $2^{50} \approx 1.12 \times 10^{15}$ candidate solutions; testing one solution every microsecond would require more than 35,000 years.

Dynamic programming avoids this combinatorial explosion by formulating the problem recursively. Let the items be indexed $0, 1, \dots, n-1$.

Definition 2.3 Knapsack Subproblem State

Let $O(j, k)$ denote the optimal (maximum) profit that can be obtained by selecting a subset from the prefix of items $\{0, 1, \dots, j\}$ subject to a maximum capacity of k :

$$O(j, k) = \max_{\mathbf{x} \in \{0, 1\}^{j+1}} \left\{ \sum_{i=0}^j v_i x_i \mid \sum_{i=0}^j w_i x_i \leq k \right\}. \quad (2.7)$$

Our ultimate goal is to determine $O(n-1, C)$.

To compute $O(j, k)$, we consider the binary decision on item j :

1. **Do not select item j** ($x_j = 0$): The available capacity remains k , and the best achievable profit using the remaining items $\{0, \dots, j-1\}$ is $O(j-1, k)$.
2. **Select item j** ($x_j = 1$): This decision is feasible only if $w_j \leq k$. If chosen, it consumes w_j units of capacity and provides profit v_j . The remaining capacity is $k - w_j$, and the optimal profit on the remaining items $\{0, \dots, j-1\}$ is $O(j-1, k - w_j)$, yielding total profit $v_j + O(j-1, k - w_j)$.

Bellman's Principle of Optimality dictates that we simply take the maximum over these two options.

Theorem 2.2 Bellman Recurrence for 0/1 Knapsack

For any item index $j \in \{0, \dots, n-1\}$ and capacity $k \in \{0, \dots, C\}$:

$$O(j, k) = \begin{cases} 0 & \text{if } j < 0 \text{ or } k \leq 0, \\ O(j-1, k) & \text{if } w_j > k, \\ \max(O(j-1, k), v_j + O(j-1, k - w_j)) & \text{if } w_j \leq k. \end{cases} \quad (2.8)$$

2.3 Naive Recursive Implementation and Time Complexity

Directly translating the recurrence relation (2.8) into Java yields the recursive procedure shown in Listing 2.1.

```
1 public class KnapsackNaive {
2     int[] value;
3     int[] weight;
4     int capa;
5     int calls = 0; // counter to track recursive invocations
6
7     public KnapsackNaive(int[] value, int[] weight, int capa) {
8         this.value = value;
9         this.weight = weight;
10        this.capa = capa;
11    }
12
13    public int solve() {
14        return opt(value.length - 1, capa);
15    }
16
17    /**
18     * Best solution on items [0..j] with residual capacity k.
19     */
20    private int opt(int j, int k) {
21        calls++;
22        if (j < 0) {
23            return 0;
24        } else if (weight[j] > k) {
25            return opt(j - 1, k);
26        } else {
27            int optWithout = opt(j - 1, k); // do not take
28            // item j
29            int optWith = value[j] + opt(j - 1, k - weight[j]); // take
30            // item j
31            return Math.max(optWithout, optWith);
32        }
33    }
34 }
```

Listing 2.1: Naive recursive implementation of 0/1 Knapsack in Java (`KnapsackNaive.java`).

2.3.1 Complexity Analysis of the Naive Recursion

Let $T(j)$ denote the number of calls required to compute $\text{opt}(j, k)$ in the worst case (when $w_i \leq k$ for all i):

$$T(j) = 2T(j - 1) + \mathcal{O}(1), \quad T(0) = 1. \quad (2.9)$$

Solving this recurrence reveals that the recursion tree is a full binary tree of depth n :

$$T(n) = \sum_{\ell=0}^n 2^\ell = 2^{n+1} - 1 = \Theta(2^n). \quad (2.10)$$

The naive recursion re-computes identical subproblems an exponential number of times.

2.4 Overlapping Subproblems and Memoization

Why does the naive recursion take exponential time? The reason is *overlapping subproblems*.

Consider the following concrete instance:

$$\begin{aligned} \text{Values } v &= [1, 6, 18, 22, 28] \\ \text{Weights } w &= [2, 3, 5, 6, 7] \\ \text{Capacity } C &= 11. \end{aligned} \tag{2.11}$$

Let us observe two different decision paths from the root state ($j = 4, k = 11$):

- Path A: Do not take item 4, take item 3 ($w_3 = 6$), and do not take item 2 ($w_2 = 5$). The consumed weight is 6, leaving residual capacity $11 - 6 = 5$. The subproblem to solve is now $\text{opt}(1, 5)$.
- Path B: Do not take item 4, do not take item 3, but take item 2 ($w_2 = 5$). The consumed weight is again 5, leaving residual capacity $11 - 5 = 6$. Looking deeper, choosing items 0 and 1 ($w_0 + w_1 = 2 + 3 = 5$) also leaves residual capacity $11 - 5 = 6$.

Whenever two execution branches arrive at the same item index j with the **exact same remaining capacity** k , the subtrees rooted at those calls are identical. Solving them more than once is completely redundant.

2.4.1 Memoization with a Hash Map

The core idea of dynamic programming is **memoization**: storing the result of $\text{opt}(j, k)$ the first time it is computed, and looking it up in $\mathcal{O}(1)$ time whenever it is encountered again.

Listing 2.2 shows the implementation using a Java `HashMap` where the key is a pair (j, k) .

```

1  import java.util.HashMap;
2  import util.Pair;
3
4  public class KnapsackHash {
5      int[] value;
6      int[] weight;
7      int capa;
8      HashMap<Pair<Integer, Integer>, Integer> cache;
9
10     public KnapsackHash(int[] value, int[] weight, int capa) {
11         this.value = value;
12         this.weight = weight;
13         this.capa = capa;
14         this.cache = new HashMap<>();
15     }
16
17     public int solve() {
18         return optCache(value.length - 1, capa);
19     }
20
21     private int optCache(int j, int k) {
22         Pair<Integer, Integer> entry = new Pair<>(j, k);
23         if (cache.containsKey(entry)) {
24             return cache.get(entry); // cache hit: O(1)
25         }
26         int res = opt(j, k);
27         cache.put(entry, res); // cache store
28         return res;
29     }
30
31     private int opt(int j, int k) {
32         if (j < 0) return 0;
33         if (weight[j] > k) return optCache(j - 1, k);

```

```

34     int optWithout = optCache(j - 1, k);
35     int optWith    = value[j] + optCache(j - 1, k - weight[j]);
36     return Math.max(optWithout, optWith);
37 }
38 }

```

Listing 2.2: Memoized Knapsack using a Hash Map (KnapsackHash.java).

Top-Down Sparsity: Reachable vs. Theoretical States. On our 5-item instance with $C = 11$, the Cartesian product of subproblem coordinates contains $(11 + 1) \times 5 = 60$ possible state pairs (j, k) . However, top-down memoization visits and caches **only 30 distinct states**—evaluating exactly half of the theoretical state space by exploring only subproblems reachable along valid item selection paths, while completely bypassing the exponential recursion of the naive algorithm.

2.5 Tabulation: Implementation with a 2D Array

While a `HashMap` provides an elegant top-down memoization strategy, it incurs object allocation overhead, auto-boxing, and hash collision costs. Because the state space is bounded by integer intervals $j \in \{0, \dots, n - 1\}$ and $k \in \{0, \dots, C\}$, we can store the subproblem values in a simple **2D primitive integer array**:

$$\text{table}[k][i] \quad \text{where } k \in \{0, \dots, C\}, i \in \{0, \dots, n - 1\}. \quad (2.12)$$

Here, $\text{table}[k][i]$ stores the optimal profit achievable using a subset of items from $\{0, \dots, i\}$ with maximum capacity k .

2.5.1 Iterative Algorithm (Bottom-Up)

Instead of recursion, we iteratively fill the table in topological order: from smaller item subsets to larger ones, and from smaller capacities to larger ones.

Algorithm 1: 0/1 Knapsack via 2D Array Tabulation

Input: Item values $v[0 \dots n - 1]$, weights $w[0 \dots n - 1]$, knapsack capacity C
Output: Optimal profit

```

1 Allocate 2D array  $\text{table}[0 \dots C][0 \dots n - 1] \leftarrow 0$ ;
  // Base case: item 0
2 for  $k \leftarrow w[0]$  to  $C$  do
3    $\text{table}[k][0] \leftarrow v[0]$ ;
  // Fill table item by item
4 for  $i \leftarrow 1$  to  $n - 1$  do
5   for  $k \leftarrow 0$  to  $w[i] - 1$  do
6      $\text{table}[k][i] \leftarrow \text{table}[k][i - 1]$ ;
7   for  $k \leftarrow w[i]$  to  $C$  do
8      $\text{optWithout} \leftarrow \text{table}[k][i - 1]$ ;
9      $\text{optWith} \leftarrow v[i] + \text{table}[k - w[i]][i - 1]$ ;
10     $\text{table}[k][i] \leftarrow \max(\text{optWithout}, \text{optWith})$ ;
11 return  $\text{table}[C][n - 1]$ ;

```

The Java implementation is given in Listing 2.3.

```

1 public class KnapsackTable {
2     int[] value;
3     int[] weight;

```

```

4   int capa;
5   int[][] table;
6
7   public KnapsackTable(int[] value, int[] weight, int capa) {
8       this.value = value;
9       this.weight = weight;
10      this.capa = capa;
11      // table[k][i] = best solution on items [0..i] and max capa k
12      this.table = new int[capa + 1][value.length];
13  }
14
15  public int solve() {
16      // Base case: item 0
17      for (int k = weight[0]; k <= capa; k++) {
18          table[k][0] = value[0];
19      }
20      // Fill table for items 1 to n-1
21      for (int i = 1; i < value.length; i++) {
22          for (int k = 0; k < weight[i]; k++) {
23              table[k][i] = table[k][i - 1];
24          }
25          for (int k = weight[i]; k <= capa; k++) {
26              int optWithout = table[k][i - 1];
27              int optWith    = value[i] + table[k - weight[i]][i - 1];
28              table[k][i] = Math.max(optWith, optWithout);
29          }
30      }
31      return table[capa][value.length - 1];
32  }
33
34  /**
35   * Reconstructs the optimal subset of selected items by backtracking.
36   * @return boolean array where true indicates the item is selected
37   */
38  public boolean[] reconstructSolution() {
39      boolean[] selected = new boolean[value.length];
40      int k = capa;
41      for (int i = value.length - 1; i > 0; i--) {
42          // If value differs from the cell without item i, item i was
43          // taken
44          if (table[k][i] != table[k][i - 1]) {
45              selected[i] = true;
46              k -= weight[i];
47          }
48          // Base case: item 0
49          if (k >= weight[0] && table[k][0] > 0) {
50              selected[0] = true;
51          }
52          return selected;
53      }
54  }

```

Listing 2.3: Iterative 2D table implementation of 0/1 Knapsack (KnapsackTable.java).

2.5.2 Subproblem Dependencies: The Two Incoming Edges

A fundamental observation about the 2D dynamic programming grid is that the computation of any cell $\text{table}[k][i]$ depends *exclusively* on values in the immediately preceding column $i - 1$. Specifically, every entry has **at most two incoming candidate edges**:

1. **The Horizontal Transition (Exclude Item i):** If item i is not selected ($x_i = 0$), the knapsack capacity remains k and no value is added:

$$\text{table}[k][i-1] \xrightarrow{\text{reward } 0} \text{table}[k][i]. \quad (2.13)$$

2. **The Diagonal Transition (Include Item i):** If item i is selected ($x_i = 1$), which is feasible only when $k \geq w_i$, item i consumes w_i units of capacity and contributes profit v_i :

$$\text{table}[k-w_i][i-1] \xrightarrow{\text{reward } +v_i} \text{table}[k][i]. \quad (2.14)$$

The stored value $\text{table}[k][i]$ is simply the maximum between these two incoming transitions. When the capacity is insufficient to accommodate item i ($k < w_i$), the diagonal edge is infeasible, and the cell trivially inherits its value from the horizontal predecessor $\text{table}[k][i-1]$.

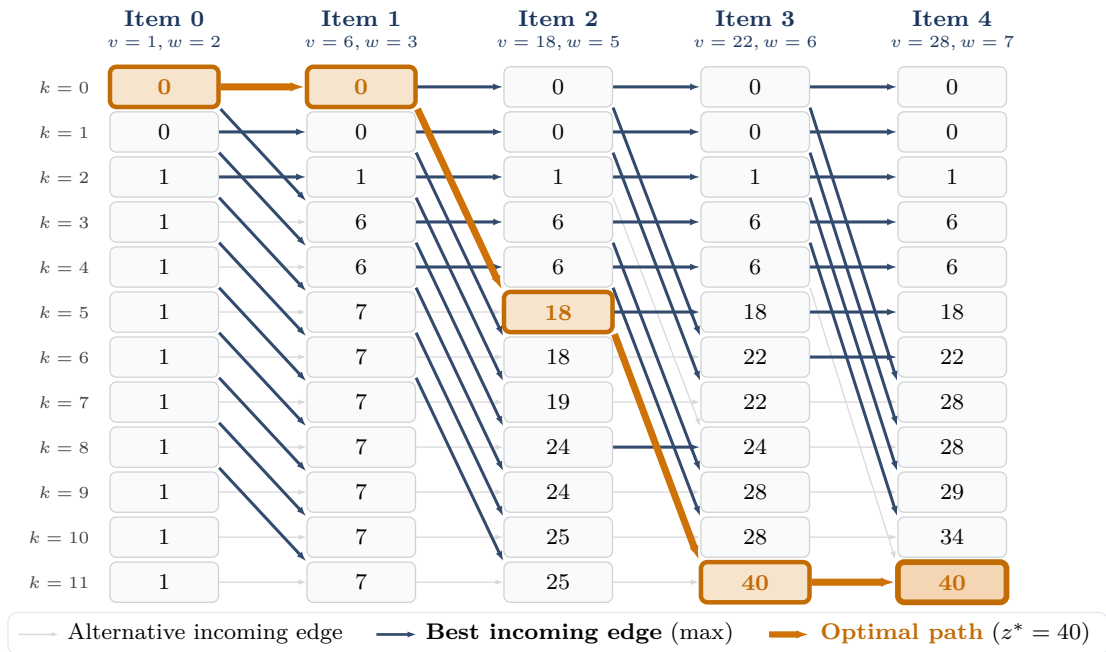


Figure 2.1: Complete 0/1 Knapsack dynamic programming table displaying all subproblem transitions across all 12×5 states. Faint gray arrows represent alternative candidate transitions; bold navy arrows indicate the **best incoming edge** (max) that determines each cell's value; the prominent amber trajectory highlights the **optimal solution path** originating at $(0, 0)$ and culminating at the optimal profit $\text{table}[11][4] = 40$.

Figure 2.1 visualizes this complete transition structure:

- **Faint gray arrows:** Transitions evaluated that did not achieve the maximum.
- **Bold navy arrows:** Winning transitions determining cell values (e.g., cell $(11, 3)$ selects the diagonal arrow from $(5, 2)$ since $18 + 22 = 40 > 25$).
- **Amber trajectory:** Tracing the winning arrows backwards from $\text{table}[11][4]$ identifies the exact **optimal path** and packed items.

2.5.3 The Tabulation Matrix Illustrated

On our running instance ($v = [1, 6, 18, 22, 28]$, $w = [2, 3, 5, 6, 7]$, $C = 11$), Table 2.1 displays the completed numerical values of the DP table. The optimal profit of **40** is found at the bottom-right corner $\text{table}[11][4]$.

Table 2.1: Dynamic Programming Table $\text{table}[k][i]$ for $C = 11$. Bold values with asterisks (*) mark the subproblems visited along the optimal solution path.

Capacity k	Item 0 (1, 2)	Item 1 (6, 3)	Item 2 (18, 5)	Item 3 (22, 6)	Item 4 (28, 7)
0	0*	0*	0	0	0
1	0	0	0	0	0
2	1	1	1	1	1
3	1	6	6	6	6
4	1	6	6	6	6
5	1	7	18*	18	18
6	1	7	18	22	22
7	1	7	19	22	28
8	1	7	24	24	28
9	1	7	24	28	29
10	1	7	25	28	34
11	1	7	25	40*	40*

2.5.4 Reconstructing the Optimal Solution via Backtracking

While the matrix entry $\text{table}[C][n - 1]$ computes the optimal numerical objective value ($z^* = 40$), practical applications require the actual binary decision vector:

$$\mathbf{x}^* = [x_0^*, x_1^*, \dots, x_{n-1}^*] \in \{0, 1\}^n. \quad (2.15)$$

Because the table was filled in forward topological order, the optimal decisions can be recovered in reverse order by **backtracking** from cell $(C, n - 1)$ back to stage 0, as formalized in Algorithm 2.

Algorithm 2: Knapsack Solution Reconstruction via Backtracking

Input: Completed DP table $\text{table}[0 \dots C][0 \dots n - 1]$, values v , weights w , capacity C
Output: Binary decision vector $\mathbf{x} \in \{0, 1\}^n$

```

1 Allocate array  $x[0 \dots n - 1] \leftarrow 0$ ;
2  $k \leftarrow C$ ;
3 for  $i \leftarrow n - 1$  to 1 step -1 do
4   if  $\text{table}[k][i] \neq \text{table}[k][i - 1]$  then
5      $x[i] \leftarrow 1$ ; // Item  $i$  was selected
6      $k \leftarrow k - w[i]$ ; // Deduct capacity consumed by item  $i$ 
7   else
8      $x[i] \leftarrow 0$ ; // Item  $i$  was skipped
  // Base case: evaluate item 0
9 if  $k \geq w[0]$  and  $\text{table}[k][0] > 0$  then
10   $x[0] \leftarrow 1$ ;
11 return  $\mathbf{x}$ ;
```

Step-by-Step Backtracking Walkthrough

Applying Algorithm 2 to our running example reveals the exact sequence of optimal decisions:

1. **Start at** ($k = 11, i = 4$): We compare $\text{table}[11][4] = 40$ with $\text{table}[11][3] = 40$. Because the values are identical, item 4 was **not** needed to achieve profit 40. We set $x_4^* = 0$, keep capacity $k = 11$, and step to (11, 3).

2. **At** ($k = 11, i = 3$): We compare $\text{table}[11][3] = 40$ with $\text{table}[11][2] = 25$. Because $40 \neq 25$, the value *must* have originated from including item 3! We set $x_3^* = 1$, deduct weight $w_3 = 6$ ($k \leftarrow 11 - 6 = 5$), and step to $(5, 2)$.
3. **At** ($k = 5, i = 2$): We compare $\text{table}[5][2] = 18$ with $\text{table}[5][1] = 7$. Because $18 \neq 7$, item 2 was selected! We set $x_2^* = 1$, deduct weight $w_2 = 5$ ($k \leftarrow 5 - 5 = 0$), and step to $(0, 1)$.
4. **At** ($k = 0, i = 1$): Here $k = 0 < w_1 = 3$. Item 1 cannot be packed. We set $x_1^* = 0$ and step to $(0, 0)$.
5. **At** ($k = 0, i = 0$): Capacity is exhausted ($k = 0 < w_0 = 2$). We set $x_0^* = 0$.

The reconstructed optimal solution is:

$$\mathbf{x}^* = [0, 0, 1, 1, 0], \quad (2.16)$$

selecting items 2 and 3 with total weight $5 + 6 = 11 \leq 11$ and total profit $18 + 22 = 40$.

Complexity of Solution Reconstruction. While computing the full dynamic programming table requires $\Theta(C \cdot n)$ time, reconstructing the optimal solution vector via backtracking requires only n comparisons and subtractions:

$$\text{Reconstruction Time} = \mathcal{O}(n), \quad \text{Auxiliary Space} = \mathcal{O}(1). \quad (2.17)$$

Solution recovery is computationally negligible compared to the table construction phase.

2.5.5 The Tabulation Grid as a DAG: The Longest Path Perspective

The observation that each cell in Figure 2.1 has incoming transitions with associated weights leads directly to a unifying theoretical concept: **the dynamic programming grid is a Directed Acyclic Graph (DAG)**, and finding the optimal solution is mathematically equivalent to finding the **Longest Path** in this DAG.

Let us define the knapsack state DAG $G = (V, E)$ explicitly:

- **Vertices V :** Every state (k, i) for $k \in \{0, \dots, C\}$ and $i \in \{0, \dots, n - 1\}$, augmented with an initial source vertex $(0, \text{Start})$.
- **Edges E :** Directed edges connecting states at stage $i - 1$ to states at stage i :
 - *Skip edge*: $(k, i - 1) \rightarrow (k, i)$ with edge weight 0 ($x_i = 0$).
 - *Take edge*: $(k - w_i, i - 1) \rightarrow (k, i)$ with edge weight $+v_i$ ($x_i = 1$, valid for $k \geq w_i$).

Because all directed edges point strictly from stage $i - 1$ to stage i , the graph cannot contain cycles: it is strictly a *layered acyclic graph*.

Notice that the dynamic programming recurrence (2.8) can be rewritten directly in graph-theoretic terms:

$$\text{table}[k][i] = \max_{(u,v) \in E, v=(k,i)} (\text{table}[u] + \text{weight}(u, v)). \quad (2.18)$$

Notice that this is *precisely* the Bellman-Ford recurrence for computing the **longest path in a Directed Acyclic Graph (DAG)** using topological ordering! Every path from the source $(0, \text{Start})$ to a terminal state represents a feasible item selection, its cumulative edge weight is the total profit, and the dynamic programming solution is simply the maximum-weight trajectory through the grid.

Keep This in Mind for Our Generic DP Solver. We invite the reader to remember this observation for later. In Section 2.8, we will formalize how *every* discrete dynamic program

reduces to a shortest or longest path problem on a DAG. Furthermore, when we design our **generic Dynamic Programming solver framework in Java** in Section 2.9, we will capitalize directly on this graph abstraction: by viewing states as graph vertices and decisions as weighted transitions, we will construct an elegant, reusable solver engine that solves DP problems without requiring hand-crafted table dimensions or hardcoded loops.

2.6 Time Complexity and Pseudo-Polynomial Time

The 2D table contains $(C + 1) \times n$ cells. Computing the value in each cell requires $\mathcal{O}(1)$ operations (one table lookup, one addition, and one comparison). Therefore:

$$\text{Time Complexity} = \Theta(C \cdot n), \quad \text{Space Complexity} = \Theta(C \cdot n). \quad (2.19)$$

2.6.1 Is this Algorithm Polynomial?

At first glance, $\Theta(C \cdot n)$ might appear polynomial because it is the product of C and n . **However, it is not polynomial in the theoretical sense.**

In computational complexity theory, the running time of an algorithm is measured as a function of the **length of the input**, denoted $|I|$, defined as the number of bits required to encode the instance:

- The number of items n requires $\mathcal{O}(\log n)$ bits.
- The weights and profits require $\mathcal{O}(\sum \log w_i + \sum \log v_i)$ bits.
- The integer capacity C is encoded in binary using:

$$L = \lceil \log_2(C + 1) \rceil \quad \text{bits}. \quad (2.20)$$

Notice that the numerical value C is exponentially related to its bit length L :

$$C \approx 2^L. \quad (2.21)$$

Expressing the runtime $\Theta(C \cdot n)$ in terms of the input bit-length L reveals:

$$\text{Runtime} = \Theta(n \cdot 2^L). \quad (2.22)$$

The algorithm is **exponential in the length of the input representing C !**

Definition 2.4 Pseudo-Polynomial Time

An algorithm for a problem whose inputs include numerical values is said to run in *pseudo-polynomial time* if its execution time is polynomial in the number of input elements n and the **numeric magnitudes** of the numbers involved, but exponential in the number of bits required to represent them.

2.6.2 Weak vs. Strong NP-Hardness

This distinction partitions **NP**-hard problems into two classes:

- **Weakly NP-Hard:** Problems (such as 0/1 Knapsack and SUBSET-SUM) that admit a pseudo-polynomial time algorithm. When data numbers are bounded by a polynomial in n (or small in practice, e.g., $C \leq 10^5$), dynamic programming solves them efficiently.

- **Strongly NP-Hard:** Problems (such as the Traveling Salesperson Problem, Vertex Cover, and Bin Packing) that remain **NP**-hard even if all numbers in the input are bounded by a polynomial in n . Strongly **NP**-hard problems cannot admit pseudo-polynomial time algorithms unless $\mathbf{P} = \mathbf{NP}$.

Practical Advice for Knapsack. If the capacity C and weights w_i share a greatest common divisor $g = \gcd(C, w_0, \dots, w_{n-1}) > 1$, dividing all weights and C by g scales down the table size without altering the optimal solution. When C is exceptionally large (e.g., $C = 10^{12}$), dynamic programming on the profit space or fully polynomial-time approximation schemes (FPTAS) can be employed.

2.7 Case Study: The Range Partitioning Problem

To reinforce the dynamic programming methodology, we now examine an optimization problem where the objective is to minimize a maximum bottleneck cost: the *Range Partitioning Problem*.

Definition 2.5 Range Partitioning Problem

Given:

1. A sequence $S = [s_0, s_1, \dots, s_{n-1}]$ of n non-negative numbers,
2. An integer $k \geq 1$,

partition S into at most k contiguous sub-ranges without reordering the elements, such that the **maximum sum over all ranges is minimized**.

Example 2.1 Range Partitioning Example

Let $S = [1, 2, 3, 4, 5, 6, 7, 8, 9]$ and $k = 3$. A feasible partition into 3 contiguous ranges is:

$$[1, 2, 3, 4, 5], \quad [6, 7], \quad [8, 9]. \quad (2.23)$$

The sums of these ranges are 15, 13, and 17, respectively. The maximum range sum is $\max(15, 13, 17) = 17$, which is the optimal solution.

2.7.1 Dynamic Programming Formulation

Let $O(i, \ell)$ denote the optimal objective value on the prefix sequence $[s_0, \dots, s_i]$ using at most ℓ partitions (with $i \in \{0, \dots, n-1\}$ and $\ell \in \{1, \dots, k\}$).

To determine $O(i, \ell)$, we make a decision: *where to place the split* between the $(\ell - 1)$ -th partition and the final (ℓ -th) partition. Suppose the last partition covers elements from index $j + 1$ to i , namely $[s_{j+1}, \dots, s_i]$:

- The sum of the last partition is $\sum_{m=j+1}^i s_m$.
- The preceding prefix $[s_0, \dots, s_j]$ is partitioned into at most $\ell - 1$ sub-ranges, yielding optimal bottleneck cost $O(j, \ell - 1)$.
- The bottleneck cost for this choice is $\max\left(O(j, \ell - 1), \sum_{m=j+1}^i s_m\right)$.

Since we wish to minimize the bottleneck, we minimize over all valid split indices $j \in \{0, \dots, i-1\}$:

$$O(i, \ell) = \min_{0 \leq j \leq i-1} \max \left(O(j, \ell - 1), \sum_{m=j+1}^i s_m \right). \quad (2.24)$$

Base Cases:

1. With only $\ell = 1$ partition:

$$O(i, 1) = \sum_{m=0}^i s_m, \quad \forall i \in \{0, \dots, n-1\}. \quad (2.25)$$

2. For a sequence of length 1 ($i = 0$):

$$O(0, \ell) = s_0, \quad \forall \ell \in \{1, \dots, k\}. \quad (2.26)$$

The optimal global solution is given by $O(n-1, k)$.

2.7.2 Prefix Sum Precomputation

Computing $\sum_{m=j+1}^i s_m$ naively takes $\mathcal{O}(n)$ time. By precomputing a *prefix sum array*:

$$\text{prefixSum}[i] = \sum_{m=0}^i s_m, \quad \text{with } \text{prefixSum}[i] = \text{prefixSum}[i-1] + s_i, \quad (2.27)$$

any range sum is obtained in $\mathcal{O}(1)$ time:

$$\sum_{m=j+1}^i s_m = \text{prefixSum}[i] - \text{prefixSum}[j]. \quad (2.28)$$

2.7.3 Java Implementation

The complete Java implementation using 2D tabulation is presented in Listing 2.4.

```

1  import java.util.Arrays;
2
3  class RangePartitioningTable {
4      int[] input;
5      int[] prefixSum;
6      int n;
7      int nPartitions;
8      int[][] cache;
9
10     public RangePartitioningTable(int[] input, int k) {
11         this.input = input;
12         this.n = input.length;
13         this.nPartitions = k;
14         // cache[k][i] = optimal solution with k partitions on input[0..i]
15         this.cache = new int[k + 1][n];
16
17         // Precompute prefix sums for O(1) interval queries
18         prefixSum = new int[n];
19         prefixSum[0] = input[0];
20         for (int i = 1; i < n; i++) {
21             prefixSum[i] = prefixSum[i - 1] + input[i];
22         }
23     }
24
25     private int sumInterval(int start, int end) {
26         return prefixSum[end] - prefixSum[start - 1];
27     }
28

```

```

29     int solve() {
30         // Base cases
31         for (int k = 1; k <= nPartitions; k++) {
32             cache[k][0] = input[0]; // k partitions on sequence of 1
                                     // element
33         }
34         for (int i = 1; i < n; i++) {
35             cache[1][i] = cache[1][i - 1] + input[i]; // 1 partition up to
                                                         // i
36         }
37
38         // Fill table: k from 2 to nPartitions, i from 1 to n-1
39         for (int k = 2; k <= nPartitions; k++) {
40             for (int i = 1; i < n; i++) {
41                 int min = Integer.MAX_VALUE;
42                 for (int j = 0; j <= i - 1; j++) {
43                     int res = Math.max(sumInterval(j + 1, i), cache[k -
44                                         1][j]);
45                     if (res < min) {
46                         min = res;
47                     }
48                 }
49                 cache[k][i] = min;
50             }
51         }
52         return cache[nPartitions][n - 1];
53     }

```

Listing 2.4: Range Partitioning with 2D Tabulation (RangePartitioningTable.java).

Theorem 2.3 *Complexity of Range Partitioning DP*

The dynamic program fills a table of size $(k + 1) \times n$. For each cell (k, i) , testing all split points $j \in \{0, \dots, i - 1\}$ takes $\mathcal{O}(i) = \mathcal{O}(n)$ time using prefix sums. Thus, the total time complexity is $\mathcal{O}(k \cdot n^2)$ and the space complexity is $\mathcal{O}(k \cdot n)$.

2.8 Dynamic Programming as Paths in a Directed Acyclic Graph

A unifying insight in discrete optimization is that **every discrete dynamic program can be reduced to a shortest or longest path problem in a Directed Acyclic Graph (DAG)**, specifically a *layered acyclic graph*.

Consider the state graph $G = (V, E)$ defined by the dynamic program:

- **Vertices V :** Each node corresponds to a reachable state $s \in \mathcal{S}$ (e.g., (j, k) for Knapsack).
- **Edges E :** A directed edge $(u, v) \in E$ corresponds to a feasible transition from state u to successor state v , with edge weight equal to the transition cost or value.
- **Source:** The root state $r \in V$ representing the unassigned problem (e.g., $(n - 1, C)$).
- **Sink:** The terminal base case state $t \in V$ (e.g., $j < 0$ or $k = 0$).

Because decisions progress strictly through stages (e.g., item indices decrease monotonically $j \rightarrow j - 1$), the graph contains no cycles: it is strictly a DAG.

Theorem 2.4 *DP Equivalence to DAG Paths*

- Any **maximization** DP problem is equivalent to finding a **Longest Path** from the root r to a sink in the state DAG.
- Any **minimization** DP problem is equivalent to finding a **Shortest Path** from the root r to a sink in the state DAG.

Why DAGs are Special. Finding the longest simple path in an arbitrary directed graph is a notorious NP-hard problem (it contains the Hamiltonian Path problem as a special case). However, on a **Directed Acyclic Graph**, topological sorting guarantees that longest and shortest paths can be solved in **linear time**:

$$\mathcal{O}(|V| + |E|). \quad (2.29)$$

Dynamic programming is precisely this linear-time traversal over the state DAG!

2.9 A Generic Dynamic Programming Framework in Java

Recognizing that every dynamic program is fundamentally a path search over a state DAG opens up an elegant software engineering opportunity: rather than writing ad-hoc loops and arrays for every new problem, we can build a **reusable, generic Dynamic Programming solver framework**.

We now present a generic framework, located in package `dynamicprogramming`. The framework is composed of four clean abstractions:

1. **State**: The nodes of the DAG.
2. **Transition<S>**: The directed edges of the DAG.
3. **Model<S>**: The declarative specification of the problem (root, transitions, base cases).
4. **DynamicProgramming<S>**: The generic solver engine that performs memoized search and solution reconstruction.

2.9.1 The State Abstraction

Every state must be hashable so that the generic solver can store it in a global hash table. In Java, this means providing implementations of `hashCode()` and `equals()`, as shown in Listing 2.5.

```

1 public abstract class State {
2     abstract int hash();
3     abstract boolean isEqual(State state);
4
5     @Override
6     public int hashCode() {
7         return this.hash();
8     }
9
10    @Override
11    public boolean equals(Object o) {
12        if (o instanceof State) {
13            return isEqual((State) o);
14        }
15        return false;
16    }
17 }
```

Listing 2.5: The State abstract class (DynamicProgramming.java).

2.9.2 The Transition Class

A transition represents a directed edge pointing to a successor state, carrying the decision identifier and the transition value (weight):

```
1 public class Transition<S extends State> {
2     private S successor;
3     private int decision;
4     private double value;
5
6     public Transition(S successor, int decision, double value) {
7         this.successor = successor;
8         this.decision = decision;
9         this.value = value;
10    }
11
12    public S getSuccessor() { return successor; }
13    public int getDecision() { return decision; }
14    public double getValue() { return value; }
15 }
```

Listing 2.6: The Transition class (DynamicProgramming.java).

2.9.3 The Model Abstraction

The problem model defines the topology of the DAG: what the root state is, how to generate outgoing transitions, when a base case is reached, and whether the problem is a maximization or minimization:

```
1 import java.util.List;
2
3 public abstract class Model<S extends State> {
4     abstract S getRootState();
5     abstract boolean isBaseCase(S state);
6     abstract double getBaseCaseValue(S state);
7     abstract List<Transition<S>> getTransitions(S state);
8     abstract boolean isMaximization();
9 }
```

Listing 2.7: The Model abstract class (DynamicProgramming.java).

2.9.4 The Generic Solver Engine

The generic solver class `DynamicProgramming<S>` implements top-down memoization using a hash map (`HashMap<State, Double>`) and reconstructs the optimal decision sequence:

```
1 import java.util.HashMap;
2 import java.util.LinkedList;
3 import java.util.List;
4 import util.Solution;
5
6 public class DynamicProgramming<S extends State> {
7     Model<S> model;
8     HashMap<State, Double> table; // memoization cache
```

```

9
10 public DynamicProgramming(Model<S> model) {
11     this.model = model;
12     this.table = new HashMap<>();
13 }
14
15 public Solution getSolution() {
16     getValueForState(model.getRootState());
17     return rebuildSolution();
18 }
19
20 private double getValueForState(S state) {
21     if (model.isBaseCase(state)) {
22         return model.getBaseCaseValue(state);
23     }
24     if (table.containsKey(state)) {
25         return table.get(state);
26     }
27
28     double best = model.isMaximization() ? -Double.MAX_VALUE :
29         Double.MAX_VALUE;
30     for (Transition<S> transition : model.getTransitions(state)) {
31         double value = getValueForState(transition.getSuccessor()) +
32             transition.getValue();
33         if (model.isMaximization()) {
34             if (value > best) best = value;
35         } else {
36             if (value < best) best = value;
37         }
38     }
39     table.put(state, best);
40     return best;
41 }
42
43 private Solution rebuildSolution() {
44     S currentState = model.getRootState();
45     double best = table.get(currentState);
46     List<Integer> decisions = new LinkedList<>();
47     double currentValue = best;
48
49     while (!model.isBaseCase(currentState)) {
50         for (Transition<S> transition :
51             model.getTransitions(currentState)) {
52             double value = getValueForState(transition.getSuccessor())
53                 + transition.getValue();
54             if (value == currentValue) {
55                 currentState = transition.getSuccessor();
56                 decisions.add(transition.getDecision());
57                 currentValue -= transition.getValue();
58                 break;
59             }
60         }
61     }
62     return new Solution(best, decisions);
63 }

```

Listing 2.8: Core evaluation logic of DynamicProgramming<S>.

2.10 Instantiating Knapsack in the Generic Framework

With the generic solver in place, implementing a new dynamic program requires only two concise classes:

1. A problem-specific state class extending `State`.
2. A problem model class extending `Model<S>`.

2.10.1 The Knapsack State

The state encapsulates the current item index and residual capacity. It computes its hash code using `Objects.hash(item, capacity)`:

```
1  import java.util.Objects;
2
3  public class KnapsackState extends State {
4      int item, capacity;
5
6      public KnapsackState(int index, int capacity) {
7          this.item = index;
8          this.capacity = capacity;
9      }
10
11     @Override
12     int hash() {
13         return Objects.hash(item, capacity);
14     }
15
16     @Override
17     boolean isEqual(State s) {
18         if (s instanceof KnapsackState) {
19             KnapsackState other = (KnapsackState) s;
20             return this.item == other.item && this.capacity ==
                other.capacity;
21         }
22         return false;
23     }
24 }
```

Listing 2.9: Knapsack State implementation (`KnapsackState.java`).

2.10.2 The Knapsack Model

The Knapsack model defines the transitions corresponding to leaving or taking each item:

```
1  import java.util.LinkedList;
2  import java.util.List;
3  import util.knapsack.KnapsackInstance;
4
5  public class Knapsack extends Model<KnapsackState> {
6      KnapsackInstance instance;
7      KnapsackState root;
8
9      public Knapsack(KnapsackInstance instance) {
10         this.instance = instance;
11         this.root = new KnapsackState(0, instance.capacity);
12     }
13
14     @Override
```

```

15     boolean isBaseCase(KnapsackState state) {
16         return state.item == instance.n || state.capacity == 0;
17     }
18
19     @Override
20     double getBaseCaseValue(KnapsackState state) {
21         return 0.0;
22     }
23
24     @Override
25     List<Transition<KnapsackState>> getTransitions(KnapsackState state) {
26         List<Transition<KnapsackState>> transitions = new LinkedList<>();
27
28         // Decision 0: Do not take the item
29         transitions.add(new Transition<>(
30             new KnapsackState(state.item + 1, state.capacity),
31             0,
32             0.0
33         ));
34
35         // Decision 1: Take the item if remaining capacity allows
36         if (instance.weight[state.item] <= state.capacity) {
37             transitions.add(new Transition<>(
38                 new KnapsackState(state.item + 1, state.capacity -
39                     instance.weight[state.item]),
40                 1,
41                 instance.value[state.item]
42             ));
43         }
44
45         return transitions;
46     }
47
48     @Override
49     KnapsackState getRootState() { return root; }
50
51     @Override
52     boolean isMaximization() { return true; }

```

Listing 2.10: Knapsack Model implementation (Knapsack.java).

To solve the knapsack instance, the user simply runs:

```

Knapsack model = new Knapsack(instance);
DynamicProgramming<KnapsackState> solver = new DynamicProgramming<>(model);
Solution solution = solver.getSolution();

```

The generic framework executes top-down memoized traversal, handles base cases, and reconstructs the optimal set of selected items seamlessly.

Student Capstone Project: The Traveling Salesperson Problem. The power of this generic DAG formulation extends far beyond knapsack. In the accompanying laboratory project, you will formulate the *Traveling Salesperson Problem* (Held-Karp algorithm) within this very framework by defining a `TSPState` consisting of the current city and a bitset of visited cities.

2.11 Exercises

Exercise 2.1 *Prefix Sum Boundary Conditions*

In Listing 2.4, method `sumInterval(start, end)` reads `prefixSum[start - 1]`. Explain what happens when `start = 0`. How should the method or prefix array be structured to handle `start = 0` cleanly without throwing an `ArrayIndexOutOfBoundsException`?

Exercise 2.2 *Memory Optimization for Knapsack DP*

Notice that computing column i in `KnapsackTable.java` requires only values from column $i - 1$. Show how to reduce the space complexity of the tabulation approach from $\mathcal{O}(C \cdot n)$ to $\mathcal{O}(C)$ using only two 1D arrays (or a single 1D array traversed backwards).

Exercise 2.3 *Partitioning with Non-Contiguous Ranges*

Show how to modify the 0/1 Knapsack recurrence equation to solve the SUBSET-SUM problem with boolean values `true/false` indicating whether a target sum is achievable.

Branch and Bound

This chapter is under preparation and will arrive later.

Linear Programming and Polyhedral Theory

This chapter is under preparation and will arrive later.

Lagrangian Relaxation

This chapter is under preparation and will arrive later.

Local Search and Metaheuristics

This chapter is under preparation and will arrive later.

Constraint Programming

This chapter is under preparation and will arrive later.

Benchmarking Optimization Algorithms

This chapter is under preparation and will arrive later.

References

- [App+06] David L Applegate et al. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2006 (cit. on p. 3).
- [Bel57] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957 (cit. on p. 13).
- [GJ79] Michael R Garey and David S Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979 (cit. on pp. 1, 13).
- [Kar72] Richard M Karp. “Reducibility among Combinatorial Problems”. In: *Complexity of Computer Computations*. Springer, 1972, pp. 85–103 (cit. on p. 14).
- [SW11] Robert Sedgewick and Kevin Wayne. *Algorithms*. 4th. Addison-Wesley Professional, 2011 (cit. on p. 10).